CS 61C
Fall 2020

Number Representation

Discussion 1: August 31st, 2020

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:



1.1 Depending on the context, the same sets of bits may represent different things.

True, it is all dependent on your interpretation.

1.2 It is possible to get an overflow error in Two's Complement when adding numbers of opposite signs.

False. 当结果超出 $[-2^{n-1}, 2^{n-1}-1]$ 时才会有 overflow.

1.3 If you interpret a N bit Two's complement number as an unsigned number, negative numbers would be smaller than positive numbers.

False, All Two's Complement negative numbers will be larger than the positive numbers.

1.4 If you interpret an N bit Bias notation number as an unsigned number (assume there are negative numbers for the given bias), negative numbers would be smaller than positive numbers.

True. $[-2^{N-1}+1, 2^{N-1}]$. $\because$ bias $= 2^{N-1}-1$.

$\Rightarrow$ negative numbers will be smaller than the positive numbers.

2 Unsigned Integers

2.1 If we have an n-digit unsigned numeral $d_{n-1}d_{n-2}\ldots d_0$ in *radix* (or *base*) r, then the value of that numeral is $\sum_{i=0}^{n-1} r^i d_i$, which is just fancy notation to say that instead of a 10's or 100's place we have an r's or r^2's place. For the three radices binary, decimal, and hex, we just let r be 2, 10, and 16, respectively.

Let's try this by hand. Recall that our preferred tool for writing large numbers is the IEC prefixing system:

Ki (Kibi) $= 2^{10}$	Gi (Gibi) $= 2^{30}$	Pi (Pebi) $= 2^{50}$	Zi (Zebi) $= 2^{70}$
Mi (Mebi) $= 2^{20}$	Ti (Tebi) $= 2^{40}$	Ei (Exbi) $= 2^{60}$	Yi (Yobi) $= 2^{80}$

(a) Convert the following numbers from their initial radix into the other two common radices:

4096 256 16 1
9 3
3 F
2 4
2 7
1 11 5

1. 0b10010011 = 147 = 0x93.
2. 63 = 0x3F = 0b00111111
3. 0b00100100 = 36 = 0x24
4. 0 = 0x0 = 0b0
5. 39 = 0x27 = 0b0010 0111 (8421)
6. 437 = 0x1B5 = 0b0001 1011 0101
7. 0x0123 = 0b0000 0001 0010 0011 = 291.

A B C D E F
10 11 12 13 14 15

(b) Convert the following numbers from hex to binary:

1. 0xD3AD = 0b 1101 0011 1010 1101
2. 0xB33F = 0b 1011 0011 0011 1111
3. 0x7EC4 = 0b 0111 1110 1100 0100

(c) Write the following numbers using IEC prefixes: IEC前缀.

- $2^{16} = 2^6 \cdot 2^{10} = 64$Ki
- $2^{27} = 2^7 \cdot 2^{20} = 128$Mi
- $2^{43} = 2^3 \cdot 2^{40} = 8$Ti
- $2^{36} = 2^6 \cdot 2^{30} = 64$Gi
- $2^{34} = 2^4 \cdot 2^{30} = 16$Gi
- $2^{61} = 2^1 \cdot 2^{60} = 2$Ei
- $2^{47} = 2^7 \cdot 2^{40} = 128$Ti
- $2^{59} = 2^9 \cdot 2^{50} = 512$Pi

(d) Write the following numbers as powers of 2:

- 2 Ki $= 2 \cdot 2^{10} = 2^{11}$
- 512 Ki $= 2^9 \cdot 2^{10} = 2^{19}$
- 16 Mi $= 2^4 \cdot 2^{20} = 2^{24}$
- 256 Pi $= 2^8 \cdot 2^{50} = 2^{58}$
- 64 Gi $= 2^6 \cdot 2^{30} = 2^{36}$
- 128 Ei $= 2^7 \cdot 2^{60} = 2^{67}$

3 Signed Integers

3.1 Unsigned binary numbers work for natural numbers, but many calculations use negative numbers as well. To deal with this, a number of different schemes have been used to represent signed numbers, but we will focus on two's complement, as it is the standard solution for representing signed integers.

- Most significant bit has a negative value, all others are positive. So the value of an n-digit two's complement number can be written as $\sum_{i=0}^{n-2} 2^i d_i - 2^{n-1} d_{n-1}$.
- Otherwise exactly the same as unsigned integers.
- A neat trick for flipping the sign of a two's complement number: flip all the bits and add 1.

- Addition is exactly the same as with an unsigned number.
- Only one 0, and it's located at 0b0.

For questions (a) through (c), assume an 8-bit integer and answer each one for the case of an unsigned number, biased number with a bias of -127, and two's complement number. Indicate if it cannot be answered with a specific representation.

(a) What is the largest integer? What is the result of adding one to that number?

1. Unsigned? $2^8-1 = 255$; 0xff add one, will overflop, the number will become zero.
2. Biased? $2^7 = 128$; 0xff add one, will become -127.
3. Two's Complement? $2^7-1 = 127$; 0xff add one. will become -128.

(b) How would you represent the numbers 0, 1, and -1?

1. Unsigned? 0b0000 0000 ; 0b0000 0001 ; can't represent -1.

bias = $2^7-1 = 127$

2. Biased? 0b01111111 (127) ; 0b10000000 (128) ; 0b 01111110 (126).
3. Two's Complement? 0b00000000 ; 0b00000001 ; 0b1111 1111

(c) How would you represent 17 and -17?

1. Unsigned? 0b 0001 0001 ; can't represent -17.
2. Biased? (144) 0b1001 0000 ; (110) 0b01101110.
3. Two's Complement? 0b0001 0001 ; 0b1110 1111

(d) What is the largest integer that can be represented by *any* encoding scheme that only uses 8 bits?

There is no such integer.

每个表示方法的范围不同.
^
数值

(e) Prove that the two's complement inversion trick is valid (i.e. that x and $\overline{x}+1$ sum to 0).

Prove: $x + \overline{x} + 1$

= 11111111 +1 = 0. (overflow).

⇒ 得证

(f) Explain where each of the three radices shines and why it is preferred over other bases in a given context.

Demical: For human hand calculations.

Binary: For computers.

Hexademical: a convenient shorthand for displaying binary numbers.

one hex digit corresponds to four binary digits.

4 Arithmetic and Counting

4.1 Addition and subtraction of binary/hex numbers can be done in a similar fashion as with decimal digits by working right to left and carrying over extra digits to the next place. However, sometimes this may result in an overflow if the number of bits can no longer represent the true sum. Overflow occurs if and only if two numbers with the same sign are added and the result has the opposite sign.

(a) Compute the decimal result of the following arithmetic expressions involving 6-bit Two's Complement numbers as they would be calculated on a computer. Do any of these result in an overflow? Are all these operations possible?

011001
−000111
010010

1. 0b011001 − 0b000111 = 0b 010 010 = 18. , No overflow.

100011
+111010
101101

2. 0b100011 + 0b111010 = 0b 011101 = 29 (overflow) ↘ Since we add two negative numbers and end up with a positive number, this results in an overflow.

3. 0x3B + 0x06 = 0b 0011 1011 + 0b 0000 0110 = 0b 0000 0001 = 1 ↘ Despite the extra truncated bit, this is not an overflow as −5 + 6 = 1.

111011
+000110
1000001

4. 0xFF − 0xAA impossible, cause only 6-bit, it needs 8-bit.

(b) What is the least number of bits needed to represent the following ranges using any number representation scheme.

1. 0 to 256 9 bits. (ranging 257 numbers). (cause 8 bits = 2^8 = 256 numbers).
2. -7 to 56 Ranging 64 numbers, so 6 bits.
3. 64 to 127 and -64 to -127 (128 nums) 7 bits.
4. Address every byte of a 12 TiB chunk of memory

need $2^4 \cdot 2^{40}$ numbers

⇒ it will be 44 bits.